

Master Computer Networks & IoT Systems

Operating System and Computer Architecture

Process Management

Sami Taktak

sami.taktak@cnam.fr

November, 2025

Introduction

Objectives

In this 3-hour lab session, you will:

- Understand process creation using `fork()`
- Learn to manage zombie and orphan processes
- Master the `wait()` family of system calls
- Implement inter-process communication using pipes
- Handle signals, particularly `SIGPIPE`
- Build a simple shell using `fork()`, `exec()`, and `wait()`

Prerequisites

- Basic C programming knowledge
- Understanding of processes and system calls from the lecture
- Access to a Linux system (Ubuntu, Debian, or similar)
- GCC compiler installed

Compilation and Execution

To compile your programs, use:

```
gcc -Wall -o program_name program_name.c
```

To run:

```
./program_name
```

1 Process Creation with fork()

1.1 Understanding fork()

The `fork()` system call creates a new process by duplicating the calling process. The child process is nearly identical to the parent but has a different PID.

Exercise 1 Simple Fork

Write a program `simple_fork.c` that:

1. Creates a child process using `fork()`
2. In the child process: prints "I am the child" infinitely
3. In the parent process: prints "I am the parent" infinitely
4. Handles the case where `fork()` fails

Starter code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void) {
6     pid_t pid;
7
8     pid = fork();
9
10    if (pid == -1) {
11        perror("fork failed");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        // TODO: Child process code
17    } else {
18        // TODO: Parent process code
19    }
20
21    return EXIT_SUCCESS;
22 }
```

Questions:

1. Compile and run the program. What do you observe?
2. Use `ps -la` in another terminal to see the process tree. What do you notice about PIDs and PPIDs?
3. Press `Ctrl-C`. What happens? Why?

Exercise 2 Process Identification

Write a program `pid_info.c` that:

1. Creates a child process
2. In the child: prints its PID and its parent's PID (PPID)

3. In the parent: prints its own PID and the child's PID
4. The parent waits for the child to terminate

Use:

- `getpid()` to get the current process ID
- `getppid()` to get the parent process ID
- `wait(NULL)` to wait for the child

Expected output example:

Parent: My PID is 1234, Child PID is 1235

Child: My PID is 1235, Parent PID is 1234

Exercise 3 Multiple Children

Write a program `multi_fork.c` that creates 5 child processes. Each child should:

1. Print its PID
2. Sleep for a random time (1-5 seconds)
3. Exit with a status code equal to its position (1st child exits with 1, 2nd with 2, etc.)

The parent should wait for all children and print their exit status.

Hint: Use a loop to create children and another loop to wait for them.

2 Zombie and Orphan Processes

2.1 Understanding Zombie Processes

A zombie process is a process that has terminated but whose exit status has not yet been collected by its parent using `wait()`.

Exercise 4 Creating a Zombie

Modify the program from Exercise 1.1 to create a zombie:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void) {
6     pid_t pid;
7
8     pid = fork();
9
10    if (pid == -1) {
11        perror("fork failed");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        // Child terminates quickly
17        printf("Child process exiting\n");
18        exit(EXIT_SUCCESS);
19    }
```

```
19 } else {  
20     // Parent does NOT call wait() and continues running  
21     printf("Parent process running...\n");  
22     for (;;) {  
23         sleep(1);  
24     }  
25 }  
26  
27 return EXIT_SUCCESS;  
28 }
```

Tasks:

1. Compile and run this program
2. In another terminal, run `ps -la` or `ps aux | grep defunct`
3. Observe the zombie process (state Z or <defunct>)
4. Kill the parent process using `kill -9 <parent_pid>`
5. Run `ps -la` again. What happened to the zombie?

Questions:

1. What is the state of the zombie process in the `ps` output?
2. Why does killing the parent remove the zombie?
3. What process adopts the zombie after the parent dies?

Exercise 5 Preventing Zombies

Modify the zombie program to prevent the zombie by adding a `wait()` call in the parent.

Tasks:

1. Add `wait(NULL);` in the parent after the child is created
2. Run the program and verify no zombie is created
3. Use `ps -la` to confirm

2.2 Understanding Orphan Processes

An orphan process is created when the parent terminates before the child. The orphan is adopted by the `init` process (PID 1).

Exercise 6 Creating an Orphan

Write a program `orphan.c`:

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3 #include <unistd.h>  
4  
5 int main(void) {  
6     pid_t pid;  
7  
8     pid = fork();  
9 }
```

```
10  if (pid == -1) {
11      perror("fork failed");
12      exit(EXIT_FAILURE);
13  }
14
15  if (pid == 0) {
16      // Child continues running
17      printf("Child: My PID is %d, PPID is %d\n",
18             getpid(), getppid());
19      sleep(5);
20      printf("Child: After sleep, my PPID is %d\n",
21             getppid());
22      exit(EXIT_SUCCESS);
23  } else {
24      // Parent terminates quickly
25      printf("Parent: My PID is %d, exiting\n", getpid());
26      exit(EXIT_SUCCESS);
27  }
28
29  return EXIT_SUCCESS;
30 }
```

Questions:

1. What is the child's PPID before the parent exits?
2. What is the child's PPID after the parent exits?
3. Who is the new parent of the orphan?
4. Is an orphan process a problem? Why or why not?

Exercise 7 Understanding wait() Return Values

Write a program that creates a child which exits with status 42. The parent should:

1. Use `wait(&status)` to retrieve the exit status
2. Use `WIFEXITED(status)` to check if normal termination
3. Use `WEXITSTATUS(status)` to extract the exit code
4. Print the exit code

```
1  #include <sys/wait.h>
2
3  int status;
4  pid_t child_pid = wait(&status);
5
6  if (child_pid == -1) {
7      perror("wait");
8      exit(EXIT_FAILURE);
9  }
10
11  if (WIFEXITED(status)) {
12      printf("Child exited with status %d\n",
13             WEXITSTATUS(status));
14  }
```

3 Program Execution with exec()

3.1 Understanding exec()

The `exec()` family of functions replaces the current process image with a new program. The process ID remains the same.

Exercise 8 Simple exec()

Write a program `simple_exec.c` that:

1. Prints "Before exec"
2. Uses `execl("/bin/ls", "ls", "-l", NULL);` to execute `ls`
3. Prints "After exec" (this should never execute)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void) {
6     printf("Before exec: PID = %d\n", getpid());
7
8     // Execute ls -l
9
10    printf("After exec: PID = %d\n", getpid());
11    perror("exec");
12    exit(EXIT_FAILURE);
13 }
```

Questions:

1. Does "After exec" get printed? Why or why not?
2. What happens to the process memory after `exec()`?

Exercise 9 fork() + exec()

Write a program `fork_exec.c` that:

1. Forks a child process
2. In the child: executes `/bin/ls -l` using `execl()`
3. In the parent: waits for the child and prints "Child finished"

```
1 if (pid == 0) {
2     // Child
3     // Execute ls -l
4     perror("exec failed"); // Only executes if exec fails
5     exit(EXIT_FAILURE);
6 } else {
7     // Parent
8     // Wait for the child
9     printf("Child finished\n");
10 }
```

4 Inter-Process Communication with Pipes

4.1 Anonymous Pipes

Pipes provide a unidirectional communication channel between processes.

Exercise 10 Simple Pipe Communication

Write a program `simple_pipe.c` where:

1. Parent creates a pipe
2. Parent forks a child
3. Parent sends a message to the child through the pipe
4. Child reads and prints the message

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5
6 int main(void) {
7     int pipefd[2];
8     pid_t pid;
9     char buffer[128];
10    const char *msg = "Hello from parent!";
11
12    // Create pipe
13
14    // Fork a child
15
16    if (pid == 0) {
17        // Child: close write end, read from pipe
18
19        // Child: read a message from Parent
20        buffer[n] = '\0';
21        printf("Child received: %s\n", buffer);
22
23        close(pipefd[0]);
24        exit(EXIT_SUCCESS);
25    } else {
26        // Parent: close read end, write to pipe
27
28        // Parent: send a message to Child
29
30        close(pipefd[1]);
31        wait(NULL);
32    }
33
34    return EXIT_SUCCESS;
35 }
```

Important notes:

- `pipefd[0]` is the read end
- `pipefd[1]` is the write end

- Always close unused ends in each process

Exercise 11 Bidirectional Communication

Write a program where:

1. Process A sends "Hello, I am process A" to process B
2. Process B replies with "Hello, I am process B"

Hint: You need two pipes - one for each direction.

```
1 int pipe_a_to_b[2]; // For A -> B
2 int pipe_b_to_a[2]; // For B -> A
3
4 pipe(pipe_a_to_b);
5 pipe(pipe_b_to_a);
```

Exercise 12 Pipe with exec()

Create a program that:

1. Creates a pipe
2. Forks a child
3. Child redirects stdout to the pipe write end using `dup2()`
4. Child executes `ls -l`
5. Parent reads from the pipe and counts the number of lines

```
1 if (pid == 0) {
2     close(pipefd[0]); // Close read end
3
4     // Redirect stdout to pipe write end
5     dup2(pipefd[1], STDOUT_FILENO);
6     close(pipefd[1]);
7
8     execlp("ls", "ls", "-l", NULL);
9     perror("exec failed");
10    exit(EXIT_FAILURE);
11 }
```

4.2 Broken Pipe Behavior

Exercise 13 Understanding Broken Pipes

Write a program that:

1. Creates a pipe
2. Forks a child
3. Child closes both ends of the pipe and exits
4. Parent tries to write to the pipe repeatedly

Expected behavior: The parent should receive a `SIGPIPE` signal and terminate.

5 Signal Handling

5.1 Understanding SIGPIPE

When a process writes to a pipe that has no readers, the kernel sends a SIGPIPE signal. The default action is to terminate the process.

Exercise 14 Handling SIGPIPE

Modify the program from Exercise 4.4 to handle SIGPIPE:

```

1 #include <signal.h>
2
3 volatile sig_atomic_t got_signal = 0;
4
5 void sigpipe_handler(int sig) {
6     got_signal = sig; // Just set a flag
7 }
8
9 int main(void) {
10     struct sigaction sa;
11
12     // Setup signal handler
13     sa.sa_handler = sigpipe_handler;
14     sigemptyset(&sa.sa_mask);
15     sa.sa_flags = 0;
16
17     if (sigaction(SIGPIPE, &sa, NULL) == -1) {
18         perror("sigaction");
19         exit(EXIT_FAILURE);
20     }
21
22     // ... rest of pipe code ...
23 }
```

Questions:

1. What happens when you handle SIGPIPE?
2. What is the return value of `write()` when the pipe is broken?
3. How can you detect a broken pipe without using signals?

Exercise 15 SIGCHLD Handler

Write a program that:

1. Sets up a SIGCHLD handler
2. Creates 5 child processes that exit after random delays
3. Handler reaps children using `waitpid()` with `WNOHANG`
4. Parent sleeps infinitely

```

1 void sigchld_handler(int sig) {
2     // Reap all terminated children
3 }
```

Important: Use `WNOHANG` to avoid blocking, and loop to reap all terminated children.

6 Building a Simple Shell

6.1 Shell Architecture

A shell is a program that:

1. Displays a prompt
2. Reads a command line from the user
3. Parses the command and arguments
4. Forks a child process
5. Child executes the command using `exec()`
6. Parent waits for the child to complete
7. Displays the exit status
8. Repeats

Exercise 16 Basic Shell

Create a program `myshell.c` with the following structure:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6
7 #define MAX_LINE 1024
8 #define MAX_ARGS 64
9
10 int main(void) {
11     char line[MAX_LINE];
12     char *args[MAX_ARGS];
13
14     while (1) {
15         // Display prompt
16         printf("myshell> ");
17         fflush(stdout);
18
19         // Read command line with fgets
20
21         // Remove newline
22         line[strcspn(line, "\n")] = '\0';
23
24         // Skip empty lines
25         if (strlen(line) == 0) {
26             continue;
27         }
28
29         // Check for exit command
30
31         int argc = 0;
32         char *token = strtok(line, " \t");
33         while (token != NULL && argc < MAX_ARGS - 1) {
34             args[argc++] = token;
```

```

35     token = strtok(NULL, " \t");
36 }
37 args[argc] = NULL;
38
39 // Fork and execute
40 pid_t pid = fork();
41
42 if (pid == -1) {
43     perror("fork");
44     continue;
45 }
46
47 if (pid == 0) {
48     // Child process
49     // execute the command
50     perror("exec");
51     exit(EXIT_FAILURE);
52 } else {
53     // Parent process
54     int status;
55     // Wait for Child
56     // Print Child exist status
57 }
58 }
59
60 printf("Goodbye!\n");
61 return EXIT_SUCCESS;
62 }

```

Test your shell with:

- ls
- ls -l
- pwd
- echo Hello World
- cat /etc/passwd
- exit

Exercise 17 Adding Built-in Commands

Extend your shell to support built-in commands (commands that run in the shell process itself):

1. cd <directory>: Change directory using chdir()
2. mkdir <directory>: Make directory using mkdir()

```

1 // Check for built-in commands before forking
2 if (strcmp(args[0], "cd") == 0) {
3     if (argc < 2) {
4         fprintf(stderr, "cd: missing argument\n");
5     } else if (chdir(args[1]) == -1) {
6         perror("cd");
7     }

```

```
8     continue; // Don't fork
9 }
```

Exercise 18 Background Processes (Optional)

Add support for background execution using `&`:

- If the command ends with `&`, don't wait for the child
- Set up a `SIGCHLD` handler to reap background processes

```
1 // Check for background execution
2 int background = 0;
3 if (argc > 0 && strcmp(args[argc-1], "&") == 0) {
4     background = 1;
5     args[--argc] = NULL;
6 }
7
8 // In parent
9 if (!background) {
10     waitpid(pid, &status, 0);
11 } else {
12     printf("[Background process %d]\n", pid);
13 }
```

7 Additional Challenges (Bonus)

For students who finish early:

1. **Producer-Consumer:** Implement a producer-consumer pattern using pipes
2. **Shell Pipes:** Implement pipe support in your shell (e.g., `ls | grep txt`)
3. **Shell Redirection:** Implement I/O redirection (`>`, `<`, `»`)
4. **Signal Relay:** Create a chain of processes that relay signals to each other

Debugging Tips

Process Monitoring

Useful commands:

- `ps aux`: Show all processes
- `ps -la`: Show processes with state
- `pstree`: Show process tree
- `top` or `htop`: Interactive process monitor
- `kill -l`: List all signals
- `kill -SIGNAL PID`: Send signal to process

Common Errors

1. **Zombie processes:** Always call `wait()` or handle `SIGCHLD`
2. **Broken pipes:** Check `write()` return value, handle `SIGPIPE`
3. **Forgetting to close pipe ends:** Can cause deadlocks
4. **Not checking system call return values:** Always check for `-1` and use `perror()`
5. **Buffer overflows:** Always bounds-check when reading into buffers

Resources

Man Pages

Essential man pages to consult:

- `man 2 fork`
- `man 2 wait`
- `man 2 waitpid`
- `man 3 exec`
- `man 2 pipe`
- `man 2 dup2`
- `man 2 signal`
- `man 2 sigaction`
- `man 2 shmget`
- `man 2 shmat`
- `man 3 pthread_mutex_init`

Online Resources

- <https://man7.org/linux/man-pages/> - Linux man pages online
- <https://beej.us/guide/bgipc/> - Beej's Guide to IPC

Good luck and happy coding!